Convex function interpolation.py

James Evans

07/22/2025

Note: `compute_beta_j_primal` and `compute_beta_star_primal` are alternative but mathematically equivalent implementations of Tony's `linear_programming.py` program located in the Interpolation Problem subfolder.

```
35
36  def compute_beta_j_dual(points, values1, values2, j):
37      N, d = points.shape
38
39      delta_x = points - points[j]         # shape (N, d)
40      delta_y = values1 - values1[j]       # shape (N,)
41      delta_yprime = values2 - values2[j]  # shape (N,)
42
43      lam = cp.Variable(N, nonneg=True)
44
45      # Constraints:
46      constraints = []
47
48      # sum_i λ_i * (x_i - x_j) = 0
49      for k in range(d):
50          constraints.append(cp.sum(cp.multiply(lam, delta_x[:, k])) == 0)
51
52      # sum_i λ_i * (y'_i - y'_j) = 1
53      constraints.append(cp.sum(cp.multiply(lam, delta_yprime)) == 1)
54
55      # Objective: minimize sum_i λ_i * (y_i - y_j)
56      objective = cp.Minimize(cp.sum(cp.multiply(lam, delta_y)))
57
58      prob = cp.Problem(objective, constraints)
59      prob.solve()
60
61      return prob.value
62
```

What is a Directional Derivative?

Suppose $f : \mathbb{R}^d \to \mathbb{R}$ is a scalar-valued function. The **directional derivative** of f tells us how fast the function is increasing or decreasing at a point, but *specifically in a given direction.*

To build intuition, imagine you are hiking on a mountain where $f(x_1, x_2)$ represents the height at a location (x_1, x_2). If you are standing at a point $x_0 = (1, 2)$, then the slope you feel under your feet will depend on the direction you walk—north, east, or northeast. The directional derivative quantifies this slope in any given direction.

Intuitive Definition

Let $x \in \mathbb{R}^d$ be a point, and let $v \in \mathbb{R}^d$ be a direction vector. Then the directional derivative of f at x in the direction v is defined as:

$$D_v f(x) = \lim_{\varepsilon \to 0} \frac{f(x + \varepsilon v) - f(x)}{\varepsilon}$$

This measures the instantaneous rate of change of f as you move a small amount from x in the direction v.

Relation to the Gradient

If f is differentiable and $\nabla f(x) \in \mathbb{R}^d$ is its gradient at point x, then the directional derivative in direction v is given by the dot product:

$$D_v f(x) = \nabla f(x) \cdot v$$

This expresses how much of the gradient points in the direction v. The gradient itself tells you the direction of steepest ascent, while the directional derivative measures how steep the function is in a direction you care about.

In the Code Context

In the function `compute_beta_j_dual`, we are given values $y_i = f(x_i)$ and scalar derivatives y_i'. These scalars are not full gradients but are instead interpreted as:

$$y_i' = D_v f(x_i)$$

for some fixed direction $v \in \mathbb{R}^d$. That is, at each sample point x_i, we only know how the function changes in one direction. We do not know the full vector $\nabla f(x_i)$.

def compute_beta_j_dual()

Given a collection of points $x_1, \ldots, x_N \in \mathbb{R}^d$, associated scalar function values $y_i = f(x_i)$, and directional derivatives $y_i' = f'(x_i)$, the goal is to assess whether these samples are consistent with a convex function. Specifically, for a fixed reference point x_j, we seek to construct a dual certificate that verifies whether the observed triplet (x_j, y_j, y_j') is compatible with the existence of a convex interpolant.

Mathematical Setup

Define the differences:

$$\begin{aligned} \delta x_i &:= x_i - x_j \in \mathbb{R}^d, \\ \delta y_i &:= y_i - y_j \in \mathbb{R}, \\ \delta y_i' &:= y_i' - y_j' \in \mathbb{R}. \end{aligned}$$

Let $\lambda \in \mathbb{R}^N_{\geq 0}$ be a nonnegative weight vector. The optimization problem considered is:

$$\begin{aligned}\beta_j^* := \min_{\lambda \in \mathbb{R}^N_{\geq 0}} \quad & \sum_{i=1}^N \lambda_i \delta y_i \\ \text{subject to} \quad & \sum_{i=1}^N \lambda_i \delta x_i = 0 \quad \text{(centering constraint)}, \\ & \sum_{i=1}^N \lambda_i \delta y_i' = 1 \quad \text{(normalization constraint)}.\end{aligned}$$

Centering Constraint

The constraint

$$\sum_{i=1}^N \lambda_i (x_i - x_j) = 0$$

says that the weighted sum of the vectors from x_j to each x_i must cancel out. In other words, the optimization is only allowed to combine the x_i in such a way that their net effect, relative to x_j, is zero.

Why is this important? Because x_j is the reference point: it's the location where we are testing whether the function value y_j and directional derivative y_j' can be explained by nearby data under convexity. If this constraint were not present, the optimization could shift attention to a different location — not x_j — and give a result that has nothing to do with what's happening at x_j.

This constraint prevents that. It ensures that the λ_i-weighted combination of points does not move the center of analysis away from x_j. That way, the test remains anchored at x_j, and we are truly checking whether the function is behaving convexly around that specific point.

Gradient Normalization

The constraint

$$\sum_{i=1}^N \lambda_i (y_i' - y_j') = 1$$

forces the optimizer to select a direction in the space of directional derivatives. Each term $y_i' - y_j'$ tells how the slope at x_i differs from the slope at x_j. The weights λ_i decide how much each of those differences contributes.

By requiring the total weighted difference to equal 1, the optimizer is forced to commit to a specific, nontrivial direction. Without this, it could assign all-zero weights or combine the differences in a way that cancels out, which would mean no direction is actually being tested.

This constraint guarantees that slope differences are actually used, making the convexity test nontrivial and well-defined.

Objective

The objective

$$\sum_{i=1}^N \lambda_i (y_i - y_j)$$

computes a weighted sum of differences between each function value y_i and the reference value y_j. It quantifies how much higher the surrounding values are compared to y_j, according to the weights λ_i. The minimum of this expression gives the smallest such weighted discrepancy consistent with the constraints.

```
62
63 def compute_beta_star_dual(points, values1, values2):
64     N = points.shape[0]
65     beta_array = np.zeros(N)
66
67     for j in range(N):
68         beta_array[j] = compute_beta_j_dual(points, values1, values2, j)
69
70     beta_star = np.min(beta_array)
71     return beta_star
72
```

def compute_beta_star_dual()

What the Code Computes

This function evaluates the dual convexity test at every sample point x_j, using the surrounding function values and directional derivatives.

For each $j = 1, \ldots, N$, it computes a quantity β_j^* by solving a linear program centered at x_j. This program checks whether the data near x_j is locally consistent with convexity.

Once all β_j^* values are computed, the function returns

$$\beta^* = \min_j \beta_j^*.$$

This value β^* represents the smallest (most critical) result across all test points. The code is explicitly looping through every point in the dataset, running the dual test at each one individually, collecting the results, and returning the worst-case value.

Why Take the Minimum over β_j^*?

Each value β_j^* is defined as the optimal value of a minimization problem centered at a single point x_j:

$$\beta_j^* := \min_{\lambda \in \mathbb{R}^N_{\geq 0}} \sum_{i=1}^{N} \lambda_i (y_i - y_j) \quad \text{subject to constraints.}$$

This quantity measures how well the surrounding data supports convexity at x_j. If $\beta_j^* < 0$, it indicates a violation of convexity at that point. If $\beta_j^* \geq 0$, the local data is consistent with a convex function.
To evaluate whether the dataset as a whole is convex-consistent, we define

$$\beta^* := \min_j \beta_j^*.$$

This takes the worst-case value over all sample points. If $\beta^* < 0$, then convexity fails at at least one point. If $\beta^* \geq 0$, then all local tests pass.

Taking the minimum over j is therefore necessary: even a single failure invalidates convexity. The final value β^* acts as a global certificate—convexity is confirmed if and only if $\beta^* \geq 0$.

July 25, 2025

Xuan 9:31 AM

Hi, I found a problem in note book you wrote for computing betas using dual LP: there is no directional derivatives applied to the algorithm, value2 are array of values evaluated at f' (which is another convex function in tony's notebook). Sorry that I should check this earlier, but I am too busy these two days.

Kulpoe 10:33 AM

Hey, just to clarify — I'm not using any dual LP or directional derivatives in this script. The method computes the minimal β such that H_f + \beta H_g is PSD using a bisection search on the smallest eigenvalue. There are no function evaluations or directional derivatives involved — it's purely second-order via Hessians. Let me know if you're referencing a different version or problem setup.

```
import numpy as np
# This package is for symmetric matrix eigenvalues
from numpy.linalg import eigvalsh

def compute_beta_i(H_f, H_g, tol=1e-6, max_iter=100):
```

Expand min_beta.py 3 KB

Xuan 10:39 AM

I am referencing the note book you wrote for my work.

Polymath_Jr_Convex_function_interpolation_py.pdf
343.01 KB

Kulpoe 10:40 AM

Aaah. Okay. Looking at it now. Thanks for pointing this out.

Kulpoe 10:47 AM

Are you talking about this line?

```
# sum_i λ_i * (y'_i - y'_j) = 1
constraints.append(cp.sum(cp.multiply(lam, delta_yprime)) == 1)
```

You are right.

I got confused by the notation.

✅ You're Correct: values2 is not a directional derivative

From your code and comment:

```python
# values1 is array of values evaluated at f
# values2 is array of values evaluated at f'
```

This means the code is implementing:

$$\text{Check if } f + \beta f' \text{ is convex}$$

It's not using directional derivatives. Instead:

- `values1 = f(x_i)`
- `values2 = f'(x_i)` where f' is **another convex function**, not the directional derivative of f

The dual formulation:

```python
# sum_i λ_i * (y'_i - y'_j) = 1
constraints.append(cp.sum(cp.multiply(lam, delta_yprime)) == 1)
```

is simply enforcing the normalization condition for **the second function's values**, not for the derivative of f. This matches the primal objective of finding the **maximum** β such that:

$$f + \beta f' \text{ is convex}$$

❗ PDF Mistake

Your **PDF is wrong** because it says:

> "In the function `compute_beta_j_dual`, we are given values $y_i = f(x_i)$ and scalar derivatives y'_i. These scalars are not full gradients but instead interpreted as $y'_i = D_v f(x_i)$."

That is incorrect in this context.